

Optimierungsmethode – Gaussian Splat Chunking

Kernproblem:

Die Rechenleistung der Meta Quest 3S ist begrenzt, d.h., dass in der Praxis können maximal ca. 250.000 Splat-Points gleichzeitig gerendert werden, ohne dass signifikante Performanceprobleme auftreten.

Lösungsansatz:

- Komprimierung eines Gaussian Splats reduziert dessen Splat-Point-Anzahl und somit die Qualität (Verlust des angestrebten Fotorealismus)
- Daher ein hybrider Ansatz:
Ein Low-Point-Splat bildet die Grunddarstellung der Szene und gechunkter High-Point-Splat zur Darstellung eines ansehnlichen Detailgrades.

Vorgehen:

1. Gaussian Splat Komprimierung mittels vid2scene auf eine Größe von ca. 250.000 Splats (<https://vid2scene.com/autolod/>)
2. Nutzung eines Python-Skripts, um das High-Point-Splat mehrere Chunks zu unterteilen
3. Nutzung eines Python-Skripts zur Erzeugung einer .csv-Datei, welche die Metadaten und Koordinaten der einzelnen Chunks enthält
4. Importierung der PLY-Dateien (Chunks) in Unity
5. Nutzung eines C#-Skripts zur schnelleren Erstellung der Chunk-Splat-Assets, Gameobjects und Box-Collider
6. Konfiguration des Splat-Renderings in der Unity-Scene

Erfolge:

- Komprimierung der Splats funktioniert zuverlässig
- Chunking-Verfahren lässt sich grundlegend erfolgreich anwenden
- Unity-Import der Splat-Chunks funktioniert ohne Probleme
- Chunk-Rendering funktioniert im XR-Simulator von Unity

Probleme:

- Chunk-Rendering ist für die Meta Quest 3S weiterhin zu rechenintensiv
- Teilweise Überlagerung der einzelnen Chunks
- viele Splat-Points im Sichtfeld, aufgrund der Splat-Point-Dichte des Grundsplats
- Chunk-Rendering nur mit sehr stark eingeschränkten Sichtfeld performant nutzbar (ca. 20%)

Resultat:

Der Chunking-Ansatz sowie das Chunking-Rendering funktionieren grundsätzlich, erfordern jedoch weitere Optimierungen, um auf der Basis der Meta Quest 3S eine stabile Performance bei gleichzeitig hohem Detailgrad zu ermöglichen.